



Microduck 开源机器鸭 小白指南

Hugging Face 旗下 Pollen Robotics 出品 · 一只 25cm 的双足机器鸭，靠强化学习策略走路 ·
官方仓库 [pollen-robotics/microduck](#) (v0.15.1 实测) + 姊妹训练仓库 [microduck_rl](#)



\$399 (硬件整机卖)



软件与策略训练全开源



744 g (实测 MJCF 744.4 g)



15 个舵机



14 个可控关节 + 头部 4 自由度



50 Hz 控制环

一、疑问解答 (疑问即目录)

Q1 Microduck 到底是个啥？一句话说清

一句话：**一只巴掌大 (25cm)、靠“大脑” (神经网络策略) 走路的双足机器鸭**——不会扫地不会端茶，但会走、会滑轮、会踢球、用鸭嘴叼东西、摔倒了自己爬起来。它源自迪士尼“银河星际巡洋舰”里的 BDX 机器人，前身是开源项目 Open Duck Mini。

25cm

身高

744g

体重 (MJCF 实测)

15

舵机数

50Hz

控制频率

\$399

整机售价

没有它之前怎么做？ 小型双足机器人要么是几千上万元的科研平台（如 Robotis OP3 约上万元），要么是不会走路的玩具。Microduck 用“游戏级舵机 + 手机级芯片 + RL 策略”把双足行走打到了整机 \$399 的价位。

Q2 \$399 的硬件不开源，那开源的到底是什么？

这是本推文评论区争论的核心（"华强北版本" "淘宝 openduck 卖 7000"）。官方口径很清楚：**开源的是"大脑"——全部软件栈 + 策略训练管线；硬件（BOM、CAD、装配文档）不公开**。具体三块：

开源的部分	仓库 / 位置	含金量
机器人"大脑" (23 个 Rust crate)	pollen-robotics/microduck	生产级守护进程架构 + 50Hz 控制 + ONNX 策略加载 + OTA 更新系统
策略训练 (MuJoCo + PPO + sim2real)	pollen-robotics/microduck_rl	完整物理模型 (86 个 STL 网格)、训练场景、域随机化、ONNX 导出
策略分发契约 (manifest schema 2)	docs/policy-manifest.md	社区人人可发布自己的策略，机器人直接 load

推文后续更有意思：Geek Lite 又发推指出，开发者 fanhao375 拿官方公开的 MJCF 和 47 个 STL **反推出了整套装配图、CAD 装配体和电控方案并开源**——硬件闭源挡不住社区逆向。国内还有 ZeroTau（陆吾智能）的"低成本复刻方案"和义乌版 Ylduck。

Q3 为什么把"大脑"拆成 7 个守护进程？

守护进程 (daemon)：在后台常驻运行、提供持续服务的程序。Microduck 的板子 (Rockchip RK3566) 上跑 7 个，各管一摊，用 **Unix socket 上的 JSON-RPC**（一种"一行一个 JSON 请求"的进程间通话协议）互相对话：



三个设计理由，每条都值得抄走：

- **robotd 是唯一能碰电机的进程**：其他进程只发"意图" (intents, 如"朝这个速度走")，执不执行、安不安全由 robotd 的安全层裁决——权限收口，谁也绕不过安全检查。
- **三个进程故意不依赖 robotd** (configd/updaterd/btd)：控制环挂了的机器人恰恰是最需要被重新配置、更新、回滚的机器人——恢复路径不能跟被恢复对象绑死。
- **手柄、手机、脚本走同一条 JSON-RPC**：客户端可替换（今天用手柄明天用 App），而 API 每天被真实使用、不会悄悄腐烂——跟 [[MCP协议]] 解决 M×N 集成是同一个思想。

Q4 鸭子是怎么"学会"走路的？

核心是 **sim2real**：在仿真里训练，在真机上运行。四步流水线（全在 microduck_rl 仓库）：

- ① **建物理分身**：把整只鸭子的质量、惯量、碰撞体写成 **MJCF**（MuJoCo 的 XML 场景描述格式，MuJoCo 是 Google 出的物理仿真器）。**我们实测克隆了这份模型：86 个 STL 网格、82 个碰撞几何、14 个位置伺服执行器。**
- ② **强化学习训练**：**PPO**（一种策略梯度算法）在仿真里让几十只鸭子同时练走路，摔了扣分、走稳了加分；配合 **域随机化**（训练时随机改摩擦、质量、延迟）让策略到真实世界不"水土不服"。
- ③ **导出 ONNX**：训练好的神经网络导出成 **ONNX**（通用的模型交换格式），robotd 在板子上用 ONNX Runtime 加载推理。
- ④ **50Hz 上岗**：每 20 毫秒一轮——读 IMU 姿态 + 关节角 → 喂给策略 → 输出 14 个关节目标角 → 写舵机。

社区玩法：策略清单（policy manifest）schema 2 定义了 **kind**（episodic 定时返回安全姿势 / perpetual 一直跑的步态 / scripted 中途可改指令）和 **command.encoding**（constant 固定指令 / phase 相位驱动），**任何人训好一个鸭子技能，发布到 Hugging Face，一条 robotctl policy load 就能装进自己的鸭子。**

Q5 为什么非要 RL？用普通代码锁住关节不行吗？

好问题——我们直接做实验回答（见第二节实验 2/3）。**实测结论：位置伺服把关节锁成"标准站姿"，鸭子 0.4~1 秒内照样慢慢瘫下去；侧向推一把也是瘫。**因为静态锁关节没有"动态平衡"：真人站立时脚踝也在毫秒级微调重心，RL 策略就是学会了这套毫秒级微调的神经网络。定值控制能延缓倒塌，只有策略能真正站住、走起来。

Q6 MJCF 模型里有什么、没什么？（实验实锤）

仓库里其实有两份 MJCF，用途完全不同：

	kinematics 内嵌版 (robot_walk.xml)	训练仓库版 (scene.xml)
内容	只有运动链：15 关节（1 自由关节+14 铰链）+ 16 刚体 + 惯量 + 8 个 site 标记点	完整物理世界：+ 86 STL 网格 + 82 碰撞体 + 地板 + 14 个位置伺服 + INIT/STAND/SIT/FOLD 关键帧
执行器	0 个	14 个
用途	给 robotd 的 kinematics crate 算前向运动学（FK）	给 RL 训练当"物理考场"

实验 1 实锤：把 kinematics 版丢进 MuJoCo 一松手，0.24 秒掉了 0.285 米——精确等于自由落体。它连地板都没有！这教了 sim2real 第一课：**模型文件只描述鸭子本身，"世界"（地板、摩擦、接触）是训练场景额外给的。**

Q7 为什么 kinematics crate 自己写 FK，不直接用 MuJoCo?

前向运动学 (FK)：已知所有关节角度，算出脚/头等部位在空间中的位置。robotd 每秒 50 次都要问“两只脚现在在哪”（算里程计），但 MuJoCo 整个引擎是为仿真设计的，太重。于是官方用 Rust 写了个迷你 FK：**启动时把 MJCF“编译”成每条 site 的固定变换链，之后每次查询就是一次链式折叠——不查哈希表、不分配内存**。正确性靠测试钉死：`fk_against_mujoco.rs` 用 64 个随机姿态和 MuJoCo 官方结果逐点对比。**我们在 Python 里手写了同样的 FK 对拍：64 姿态 × 4 个部位，最大误差 1.8×10^{-7} 米 (0.18 微米)**——这份运动链数学是硬的。

Q8 OTA 更新为什么“整目录交换”而不是打补丁?

OTA：Over-The-Air，无线固件升级。updaterd 的流程：新版本作为**完整目录**下载到 releases/<版本号>/ → 验证签名 → 把 `current` 软链接切过去 → 重启服务 → **健康门**（问 robotd “你还好吗”）→ 不健康就**自动把旧版换回来（回滚）**，**连重启次数都有 boot counter 兜底防崩溃循环**。思想跟蓝绿发布一模一样：**发布是切换不是修改，出错永远有退路**——一个 \$399 的玩具鸭子用了比很多大厂还规矩的发布纪律。

Q9 淘宝 ¥7000 的 openduck 是它吗？华强北版靠谱吗？

评论区说的淘宝 openduck 是同一开源生态的整机商品（约 ¥7000+，比官方便宜版还贵）。华强北版的可行性：软件全开源 + 装配图已被社区反推（fanhao375）+ 国产舵机平替方案（ZeroTau）在推进，**所以“能复刻”这件事基本没有悬念；悬念在成本与调试**——15 个总线舵机、RK3566 板、IMU/ToF 模组，外加 RL 策略的 sim2real 调参。推文里“得生产的人多了才能卷下去”正是关键：开源生态降低了复刻的知识门槛，规模才能降成本门槛。

Q10 没有真机，我能学到什么、学不到什么？（诚实边界）

✅ 能：读懂一套生产级机器人软件架构（守护进程拆分 / 意图-安全层 / 健康门回滚）；跑通 MuJoCo 仿真，亲手验证 FK、惯量、伺服极限；用官方模型做二次实验；照着 `microduck_rl` 训练自己的策略（纯软件，可导出 ONNX）。

❌ 不能（本次实验的诚实限制）：本机无 Rust 工具链，未构建守护进程本体；未加载官方行走策略做“真·走路”仿真（需要 `microduck_rl` 的观测构建管线 + 训练权重）；Dynamixel 总线、BLE、相机 ISP 这些驱动层问题只有真机能暴露（官方 `simulation.md` 也是这么写明的）。

二、动手实验：给鸭子装“大脑”之前与之后（无真机，全部实测跑通）

实验素材：官方训练物理模型（`microduck_rl` 的 `scene.xml`）+ 官方 kinematics 运动链（`microduck` 内嵌 MJCF）+ MuJoCo 3.14。完整脚本：`microduck\exercise\sim_duck.py` 与 `render_duck.py`。

实验 0 • 模型体检：这份 MJCF 里到底有什么？

操作： `python sim_duck.py` (环境：Python 3.14 + `pip install mujoco`)

目的：搞清官方模型的数据构成。实现：用 MuJoCo 解析 kinematics 内嵌 MJCF 并统计。

✅ 预期结果： $nq=21$ (1 自由关节 + 14 铰链 + 四元数)、 $nu=0$ (无执行器)、 $ngeom=0$ (无碰撞体)、总质量 744.4 g —— 运动链版本"只有骨架没有肌肉和世界"。

实验 1 • 无地板坠落：sim2real 第一课

操作/目的：把模型放在 12cm 高度松手，验证"没有场景就没有世界"。每步 1ms 跑 240 步。

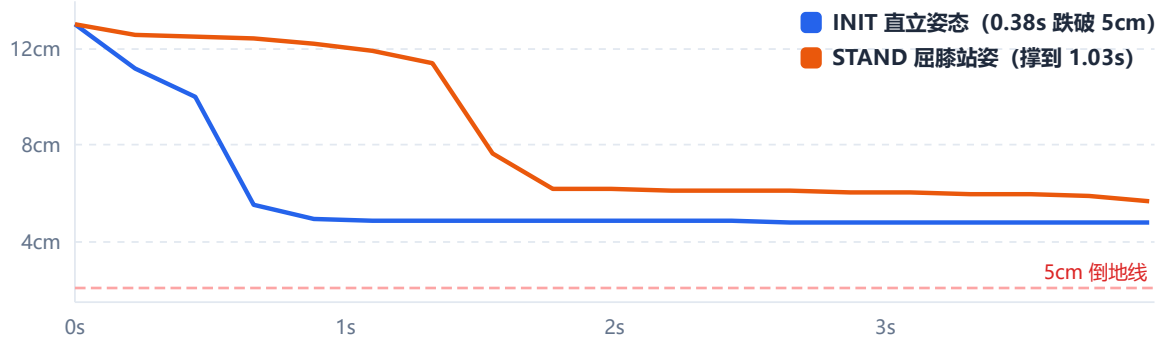
✅ 实测结果： $t=0.240s$ 时高度 $-0.165m$ ，下落 $0.285m \approx \text{自由落体 } g \cdot t^2 / 2 = 0.283m$ ，精确吻合 —— 模型文件只描述鸭子，地板/摩擦/接触全在训练场景里。

实验 2 • 最笨的"大脑"：位置伺服锁关节能站稳吗？

操作：加载完整物理场景 scene.xml，分别用官方 INIT (直立) 和 STAND (屈膝站姿) 关键帧 + 自带位置伺服跑 3 秒。目的：验证"不用 RL，普通控制行不行"。

✅ 实测结果：INIT 撑 0.38s、STAND 撑 1.03s 后高度均跌破 5cm，之后稳定瘫在 3.5~4.4cm (没散架，但站不起来) —— 好姿态 + 伺服只能延缓倒塌，不能替代动态平衡。

身体高度曲线：位置伺服锁关节 (实机同款执行器，MuJoCo 仿真)



真实仿真数据 (sim_duck.py 输出 init.csv / stand.csv, 24 点降采样)。两条线最终都瘫在 5cm 以下——没有策略，鸭子站不住。

实验 3 • 推一把：平衡为什么必须是"动态"的

操作：STAND 姿态跑仿真， $t=1s$ 时给身体 0.8 m/s 的侧向速度冲量。实现：直接改写 freejoint 的横向速度分量。

✅ 实测结果：roll 最大仅 1.8° (被伺服硬扛住)，但身体高度照样瘫到 4.4cm —— 刚性锁位扛住了倾斜，扛不住重力塌陷；真正站稳需要每 20ms 微调关节的 RL 策略。

实验 4 • 手写 FK 对拍 MuJoCo: 复刻官方测试

操作: 用纯 Python (四元数乘法 + 沿运动链折叠变换) 独立实现前向运动学, 在 64 个随机姿态下与 MuJoCo 官方 `mj_kinematics` 逐点对比 4 个部位 (left_foot / right_foot / imu / imu_bno)。**目的:** 亲手验证 microduck kinematics crate 的核心数学 (镜像上游测试 `fk_against_mujoco.rs` 的思路)。

✅ 实测结果: 256 次比对最大绝对误差 1.816×10^{-7} m。演示单点: left_knee 从 0 转到 0.6 rad, left_foot 从 $(-0.032, 0.050, 0.015)$ 移到 $(0.005, 0.050, 0.026)$ —— 膝盖一抬脚往前伸, 符合直觉。

64×4

随机姿态 × 部位

1.8×10^{-7} m

手写FK vs MuJoCo 最大误差

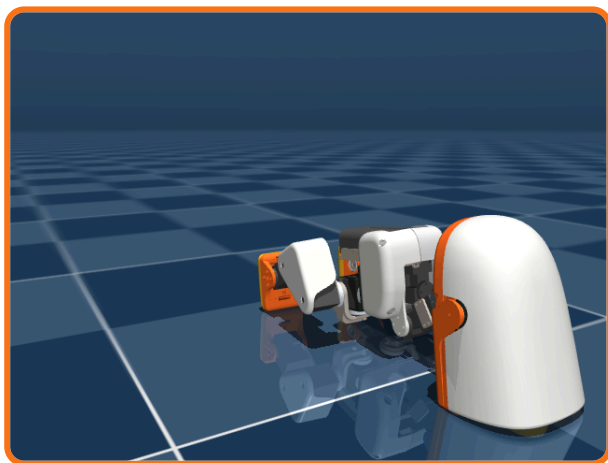
$\approx 0.18 \mu\text{m}$

比一根头发丝细 300 倍

实验 5 • 亲眼看看这只鸭: 官方模型渲染

操作: `python render_duck.py` (MuJoCo Renderer + 官方 86 个 STL 网格, STAND 关键帧)。**实现:** 仿真 0.5s 后取景渲染; 再跑满 3s 渲染瘫倒对比图。

✅ 实测结果: 下图左为 STAND 站姿 (撑过 0.5s), 右为 3s 后伺服力竭脸着地 (trunk 高度 0.12m → 0.044m), 与实验 2 曲线完全对应。



左: 官方模型 STAND 姿态仿真 0.5s (站得住) 右: 同一姿态 3s 后 (伺服无力, 脸着地) —— 均为本次实验真实渲染

三、这个项目给我什么 (能力清单)

- 🏠 **一套生产级机器人软件架构模板**：7 守护进程拆分、“意图-安全层”权限收口、恢复路径与被恢复对象解耦——任何“多进程设备/服务”项目都能抄。
- 🦆 **一条完整 sim2real 流水线**：MJCF 建模 → PPO 训练 → 域随机化 → ONNX 导出 → 50Hz 真机加载，每一环代码都公开可读（microduck_rl）。
- 🎮 **无真机的机器人实验室**：MuJoCo + 官方模型 = 零成本做动力学实验（本指南 5 个实验全是），还能接着玩官方 `scripts/duck-sim`（Linux 下仿真鸭子跑真守护进程）。
- 📦 **一份“策略即应用”的分发契约**：manifest schema 2 + Hugging Face 仓库 + `robotctl policy load`，社区技能像 App 一样装进鸭子——机器人版的 MCP 生态。
- 📘 **蓝绿发布教科书**：签名验证 → 目录交换 → 健康门 → 自动回滚 → boot counter，\$399 设备上的发布纪律。
- 🔧 **复刻/魔改入口**：软件全开源 + 社区反推装配图 + 国产舵机平替方案——想入硬件坑的人，这里是一条已被踩通的路径。

四、一页带走（结论）

Microduck = 一只“卖硬件、送大脑”的机器鸭：\$399 买舵机和板子，而真正让它站起来的秘密——守护进程架构、RL 训练管线、策略分发契约——全部开源。它证明了 2026 年做机器人 waypoints：**硬件可以闭源卖钱，护城河却在软件与 sim2real 管线；而开源的大脑一旦配合社区反推的硬件，“华强北版本”就从玩笑变成了时间表问题。**无真机能学到的部分（架构 + 仿真 + FK 数学 + 训练管线）已经足够一个工科毕业生啃上好几周，本次 5 组实验全部实测跑通。

本指南对应推文：x.com/QingQ77 (Geek Lite) “开源机器鸭，想要一个华强北版本” · 仓库 github.com/pollen-robotics/microduck (v0.15.1) 与 microduck_rl · 实验脚本 F:\reminder\microduck\exercise\ · 学习产出 2026-10-02

reminder 拾遗学习流水线 · learn-project skill 产出 · 实验数据 100% 真实运行 (Python 3.14 + MuJoCo 3.14.0, Windows 11)